Parameter-Efficient Adaptation of GPT-2 Across Classification and Generation Tasks

Varsha Saravanan, Kristine Ma, Isaias Martinez
Department of Computer Science
Stanford University
vsaravan@stanford.edu, kmayu@stanford, isaiasm@stanford.edu

Abstract

We study parameter-efficient and decoding-based extensions for adapting a shared language-model backbone across sentiment classification, paraphrase detection, and conditional sonnet generation. Starting from standard fine-tuning baselines, we compare several lightweight intervention strategies, including LoRA, ReFT, and soft prompt tuning, and we additionally examine how decoding and hyperparameter choices affect generation quality. Across tasks, we evaluate the tradeoff between downstream performance and adaptation efficiency, asking when small, targeted updates can match or exceed full-model fine-tuning. Our results show that parameter-efficient methods are competitive with standard fine-tuning and can be especially effective when intervention rank, layer placement, and training setup are chosen carefully. ReFT and LoRA both achieve statistically competitive sentiment performance with far fewer trainable parameters and memory usage (up to a 61.9% decrease in the case of ReFT on CFIMDB), while tuned decoding settings substantially improve sonnet chrF. These findings suggest that much of the practical value in downstream adaptation comes not from modifying the full model, but from choosing effective low-parameter intervention strategies.

1 Introduction

Large language models have made pretraining followed by task-specific adaptation a standard paradigm in modern NLP [1, 2]. However, adapting a pretrained model through full-model fine-tuning can require updating millions of parameters, making deployment expensive in memory, compute, and training time. This cost has motivated growing interest in parameter-efficient fine-tuning (PEFT), where only a small subset of parameters or representations is modified while the pretrained backbone remains largely frozen [3, 4, 5].

In this project, we implement a GPT-2 style decoder-only Transformer and study how it adapts across three downstream tasks: sentiment classification, paraphrase detection, and conditional sonnet generation. Rather than focusing primarily on the base architecture itself, we use GPT-2 as a common experimental backbone for comparing several adaptation strategies. In particular, we evaluate standard full-model fine-tuning alongside LoRA, ReFT, and soft prompt tuning, and we additionally study preference-based and decoding-based improvements for sonnet generation [6].

Our goal is to understand the tradeoff between downstream performance and adaptation efficiency across both classification and generation settings. We find that lightweight adaptation methods can approach, and in some cases match, the performance of full fine-tuning while using far fewer trainable parameters, and that generation quality depends not only on training strategy but also on careful decoding and reranking choices. These results suggest that much of the practical gain comes from selecting effective intervention mechanisms rather than simply updating the entire pretrained model.

Stanford CS224N Natural Language Processing with Deep Learning

2 Related Work

Decoder-only language models such as GPT and GPT-2 established the now-standard paradigm of large-scale autoregressive pretraining followed by task-specific adaptation [1, 2]. Because the same pretrained backbone can support both discriminative and generative downstream tasks, GPT-2 provides a natural foundation for studying sentiment classification, paraphrase detection, and sonnet generation within a single experimental framework. At the same time, the conventional strategy of full-model fine-tuning becomes increasingly expensive as model size grows, motivating methods that reduce the number of trainable parameters while preserving strong downstream performance.

A large body of recent work has therefore explored parameter-efficient fine-tuning (PEFT). Prompt-based methods adapt a frozen model by learning a small number of continuous prompt embeddings, allowing task-specific behavior to be induced without modifying the full backbone [3]. LoRA instead freezes the pretrained weights and learns low-rank updates inside selected linear layers, substantially reducing the number of trainable parameters while often maintaining competitive performance [4]. More recently, Representation Fine-Tuning (ReFT) proposed intervening directly on intermediate hidden representations rather than on model weights, making adaptation depend not only on parameter budget but also on the rank and placement of the intervention [5].

These methods suggest different hypotheses about where task adaptation should occur: at the input level through learned prompts, within attention projections through low-rank updates, or in intermediate representations through hidden-state interventions. Our project compares these alternatives in a unified GPT-2 setting. Rather than proposing a new backbone, we use the same pretrained decoder across multiple tasks and evaluate how full fine-tuning, LoRA, ReFT, and soft prompt tuning trade off parameter efficiency and downstream quality.

Beyond parameter efficiency, recent work on language model alignment has studied preference-based objectives that optimize models toward preferred outputs without explicit reinforcement learning. Direct Preference Optimization (DPO) is one such approach, training a policy model against a fixed reference model using paired preferred and dispreferred generations [7]. This perspective is especially relevant for open-ended generation tasks, where likelihood alone may not fully capture structural or stylistic quality. In our project, we apply this idea to sonnet generation by comparing preference-based training and decoding-based improvements against standard GPT-2 fine-tuning.

Overall, our work is most closely connected to prior research on efficient adaptation and alignment for pretrained language models. The central question we investigate is not whether GPT-2 can be adapted at all, but which lightweight intervention strategies are most effective for different downstream tasks, and when they can approach or exceed the quality of full-model fine-tuning.

3 Approach

Shared backbone. All methods use the same pretrained GPT-2 small decoder-only Transformer as a shared backbone [2]. The model follows the standard autoregressive Transformer design with learned token and positional embeddings, causal masked self-attention, residual connections, layer normalization, and feed-forward blocks [8]. We use this backbone across all tasks so that differences in performance can be attributed primarily to the adaptation method rather than to changes in architecture.

Sentiment classification baseline. For sentiment classification, we treat GPT-2 as a sentence encoder and attach a linear classification head to the final token representation. Given an input sentence, the hidden state of the last non-padding token is used as the sentence representation, followed by dropout and a linear projection to sentiment logits. We compare two standard baselines: *last-linear-layer* (LLL), where the GPT-2 backbone is frozen and only the classifier is trained, and *full-model fine-tuning*, where all parameters are updated jointly. These baselines provide reference points for evaluating whether parameter-efficient interventions can recover the benefits of full adaptation.

Paraphrase detection baseline. For paraphrase detection, we encode each question pair as a single GPT-2 input prompt and use the hidden state of the final token to predict a binary paraphrase label. A linear head maps this representation to logits for the two classes. Our baseline is full-model

fine-tuning on the Quora paraphrase task, which serves as the main comparison point for soft prompt tuning and other lightweight adaptation methods.

Sonnet generation baseline. For sonnet generation, we use GPT-2 as a standard autoregressive language model over full sonnet sequences. At each position, the model predicts the next token from the contextualized hidden state using the tied output embedding matrix, and training uses the usual next-token cross-entropy objective [2]. At evaluation time, the model is conditioned on the first three lines of a held-out sonnet and generates the continuation autoregressively. This full-model fine-tuning setup forms the baseline against which we compare parameter-efficient methods and decoding-based improvements.

Extensions.

Representation Fine-Tuning (ReFT). We evaluate Representation Fine-Tuning (ReFT), specifically a simplified low-rank variant inspired by LoReFT [5]. Unlike full-model fine-tuning, ReFT leaves the pretrained backbone frozen and instead learns task-specific interventions on intermediate hidden representations. For a hidden state $h \in \mathbb{R}^d$, the intervention is

$$h' = h + W_{\text{up}} W_{\text{down}} h,$$

where $W_{\text{down}} \in \mathbb{R}^{r \times d}$ and $W_{\text{up}} \in \mathbb{R}^{d \times r}$ define a rank-r update. This formulation makes the adaptation budget depend on both the intervention rank and the layers at which it is applied. Following standard practice, we initialize the update to preserve the pretrained model at the start of training by setting W_{up} to zero, so the initial mapping is the identity *h'=h*. In our experiments, we apply ReFT to sentiment classification and vary both intervention rank and layer placement, comparing against full fine-tuning and the frozen-backbone last-linear-layer baseline in accuracy and efficiency.

Low-Rank Adaptation (LoRA). We also evaluate Low-Rank Adaptation (LoRA), a parameter-efficient method that freezes the pretrained weights and learns low-rank updates inside selected linear transformations [4]. For a pretrained weight matrix $W \in \mathbb{R}^{d_{out} \times d_{in}}$, LoRA replaces full updating with

$$W' = W + \Delta W, \qquad \Delta W = BA,$$

where $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$ are trainable low-rank factors. The resulting forward pass is

$$y = Wx + \frac{\alpha}{r} BAx,$$

where α controls the scale of the update. We apply LoRA to the query and value projections in each attention block and keep the original GPT-2 parameters frozen. Across sentiment classification, paraphrase detection, and sonnet generation, only the LoRA parameters and task-specific output layers are trained. This provides a consistent comparison point for whether low-rank weight updates can recover the benefits of full adaptation across both classification and generation tasks.

Soft Prompt Tuning. For paraphrase detection, we experiment with soft prompt tuning, which adapts a frozen model by prepending a small set of learned continuous prompt embeddings to the input sequence [3]. Let $X \in \mathbb{R}^{T \times d}$ denote the token embeddings of the original input and let $P \in \mathbb{R}^{m \times d}$ denote m learned prompt vectors. The modified input becomes

$$\tilde{X} = [P; X].$$

The prompt embeddings participate fully in self-attention, but the pretrained backbone remains frozen. After encoding, we discard the prompt positions, recover the hidden states aligned with the original sequence, and use the final non-padding token representation for binary classification. In our experiments, we apply this method to Quora paraphrase detection and train only the soft prompt parameters together with the classification head. This setting lets us test whether input-level adaptation alone is sufficient for competitive paraphrase performance.

Direct Preference Optimization (DPO). For sonnet generation, we additionally study Direct Preference Optimization (DPO), a preference-based objective that trains a policy model against a

fixed reference model without learning a separate reward model [7]. Given a prompt x, a preferred continuation y_w, and a dispreferred continuation y_l, the loss is

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\left(\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right)\right].$$

Because the sonnet dataset does not include explicit rejected continuations, we construct them automatically using corrupted negatives. We compare three corruption strategies: line shuffling after truncation, partial replacement of continuation lines with lines from another sonnet, and full replacement of the continuation while preserving the original prompt. These variants let us test how the difficulty and structure of negative examples affect preference learning and downstream chrF.

Hyperparameter and Decoding Tuning. In addition to architectural interventions, we treat hyperparameter and decoding tuning as a controlled extension for sonnet generation. Here the training objective and model architecture remain fixed, and improvement comes from selecting optimization and decoding settings using development chrF. We perform a small learning-rate sweep for full-model fine-tuning and choose the best checkpoint by development performance. We then compare generation settings under different temperature and sampling configurations to measure how much output quality can be improved through decoding alone. This analysis is useful because, for open-ended generation, gains may come not only from the learned parameters but also from how continuations are sampled at inference time.

4 Experiments

4.1 Data

We use only the course-provided datasets and do not incorporate external data [9]. Our experiments span three settings: sentiment classification, paraphrase detection, and sonnet generation. For sentiment classification, we evaluate on SST, a 5-way sentence-level sentiment dataset, and CFIMDB, a binary movie-review sentiment dataset with longer inputs. For paraphrase detection, we use the Quora question-pairs dataset provided with the project. For conditional generation, we use the Shakespeare sonnet corpus, training on complete sonnets and evaluating by generating continuations from the first three lines of held-out prompts. As required by the project setup, model training and hyperparameter selection use only the training and development splits; test data is never used for tuning [2].

Table 1: Datasets and Size

Dataset	Task	Train	Dev	Test
SST	5-class sentiment	8,544	1,101	2,210
CFIMDB	Binary sentiment	1,701	245	488
Quora	Paraphrase detection	283,003	40,429	80,858
Sonnets	Conditional generation	131	12	12

For sonnet generation, the development and test examples are prompts consisting of the first three lines of each held-out sonnet.

4.2 Evaluation Method

We evaluate each task using the metric most directly aligned with its prediction objective. For the classification tasks, we report development-set accuracy as the primary model-selection criterion. For paraphrase detection, the model predicts a binary decision for each question pair, and we evaluate these predictions using accuracy on the Quora development set. In both cases, the best checkpoint is selected according to development accuracy. For the sentiment classification task, on which we compare LoRA and ReFT to our standard methods, we also measure efficiency (trainable parameters) and memory.

For sonnet generation, we evaluate output quality using corpus-level chrF, a character n-gram overlap metric between generated sonnet continuations and the corresponding held-out reference sonnets. At evaluation time, every sonnet model, including DPO-trained models, is conditioned on the first three

lines of each held-out sonnet and generates the remaining lines autoregressively. We compute chrF on the full set of generated continuations and use development chrF to choose the best checkpoint.

DPO differs only in the training objective, where the model is optimized on preference pairs rather than standard next-token cross-entropy. It does not change the evaluation procedure: DPO-trained models are still evaluated by autoregressive generation on held-out prompts and compared using development/test chrF.

All hyperparameter tuning and checkpoint selection are performed exclusively on the provided development splits. The test sets are used only for final prediction generation after model selection is complete. This ensures that performance comparisons across baselines and extensions reflect generalization rather than tuning on held-out test data.

4.3 Experimental Details

Unless otherwise stated, all experiments use the pretrained GPT-2 small backbone with the standard GPT-2 tokenizer and end-of-sequence padding. We train all models with AdamW, fix the random seed to 11711 for reproducibility, and run each experiment for 10 epochs. For every task, we select the checkpoint with the best development-set performance and use that checkpoint for final prediction generation. All experiments are run on GPU when available, with batch sizes chosen to satisfy memory constraints for each task.

We keep the training setup as consistent as possible across tasks while allowing task-specific hyperparameters where necessary. For sentiment classification, we apply dropout with rate 0.3 to the final token representation before the classifier head. In the frozen `last-linear-layer` setting, we use a larger learning rate 1×10^{-3}, while in `full-model` fine-tuning we reduce the learning rate to 1×10^{-5}. Additionally, whereas we typically used 64 batches when training on SST data, we used 2 batches on CFIMDB to fit GPU memory. For paraphrase detection, we use full-model fine-tuning as the main baseline with learning rate 1×10^{-5} and batch size 8; for soft prompt tuning, we keep the GPT-2 backbone frozen and train a learned prompt of length 20 initialized with standard deviation 0.02. For sonnet generation, we train with batch size 8 and compare learning rates 5×10^{-6} and 1×10^{-5}, selecting the latter based on development chrF. Unless otherwise noted, generation uses temperature 1.2, top-p sampling with $p = 0.9$, `top_k`=0, and `best_of_n`=1. For DPO sonnet experiments, we keep the same batch size and optimizer settings as the baseline while replacing the next-token prediction objective with the DPO loss.

5 Results

5.1 Sentiment Classification:

For ReFT, we varied both the intervention rank and the placement of the intervention layers. Tables 2 and 3 present an abridged comparison, with fuller results shown in the Appendix.

Our baseline full fine-tuning achieved **0.520** on SST and **0.980** on CFIMDB, whereas our baseline last linear layer achieved **0.467** on SST and **0.865** on CFIMDB.

Table 2: Impact of Rank on ReFT Performance (Layer = 5)

Rank (r)	**SST Accuracy**	**CFIMDB Accuracy**
Rank 2	0.511	0.959
Rank 4	0.504	**0.967**
Rank 8	0.509	**0.967**
Rank 16	**0.517**	0.955

Table 3: Impact of Intervention Layer Position on ReFT Performance (Rank $r = 4$)

Layer of intervention (Rank 4)	**SST Accuracy**	**CFIMDB Accuracy**
Early (Layer 2)	0.507	**0.976**
Middle (Layer 5)	0.504	0.967
Late (Layer 11)	0.480	0.873
Distributed (Layers 1, 6, 11)	**0.526**	0.971
Late cluster (Layers 8, 9, 10)	0.507	**0.976**

We see that ReFT performance is sensitive to both rank and intervention layer placement. Rank changes have a relatively modest effect on both SST and CFIMDB, though we see that a higher rank (Rank 16) for SST may be beneficial, whereas a higher rank for CFIMDB could be detrimental, due to the balance between capturing more nuances vs. overfitting.

Layer placement seems to have greater significance, especially on CFIMDB, where late-layer intervention causes a sharp drop in accuracy. On the other hand, we see consistently high performance with the Distributed layer (1, 6, 11) configuration. This suggests that sentiment is a multi-stage semantic feature. Intervening at the beginning, middle, and end of the pipeline allows the model to refine its "understanding" of the sentence as it passes through the transformer blocks.

For LoRA, we study the effect of both the adaptation rank and the scaling factor on downstream sentiment performance. Table 4 summarizes results for several representative configurations on SST and CFIMDB, highlighting how different low-rank capacities influence accuracy across datasets.

Table 4: Impact of LoRA Rank and Scaling on Sentiment Performance

Configuration	**SST Accuracy**	**CFIMDB Accuracy**
Rank 8, $\alpha = 16$	0.517	0.939
Rank 4, $\alpha = 8$	**0.519**	**0.971**
Rank 8, $\alpha = 32$	0.509	0.820
Rank 16, $\alpha = 32$	0.515	0.898

For LoRA, we observed that moderate ranks with smaller scaling factors yield the most stable results, while larger scaling values can lead to noticeable performance degradation, particularly on CFIMDB. This suggests that increasing the effective update magnitude does not always improve task adaptation and may inadvertently introduce optimization instability or overfitting effects.

In addition, for our comparisons between methods, we measure efficiency (in terms of trainable, unfrozen parameters) and compute (via peak GPU memory). Results below are shown between the highest-performing LoRA and ReFT parameters, though fuller results can be found in the Appendix.

Table 5: Efficiency, Compute, and Performance Metrics on SST

Method	**% Trainable Params**	**Peak GPU (MB)**	**Accuracy**
LLL	0.0031%	681.36	0.467
Full-model	100%	4865.96	0.520
ReFT	0.0178%	2398.49	**0.526**
LoRA	0.1209%	2861.55	0.519

Table 6: Efficiency, Compute, and Performance Metrics on CFIMDB

Method	**% Trainable Params**	**Peak GPU (MB)**	**Accuracy**
LLL	0.0012%	591.18	0.865
Full-model	100%	2710.30	**0.980**
ReFT	0.0061%	1285.82	0.976
LoRA	0.1190%	1544.81	0.971

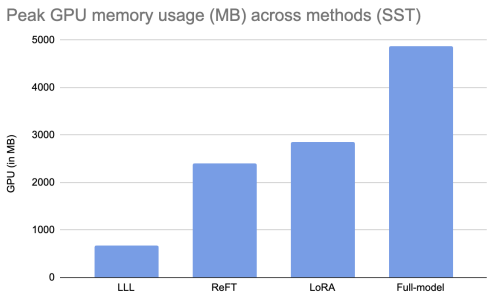


Figure 1: Visualizing memory usage across methods (SST)

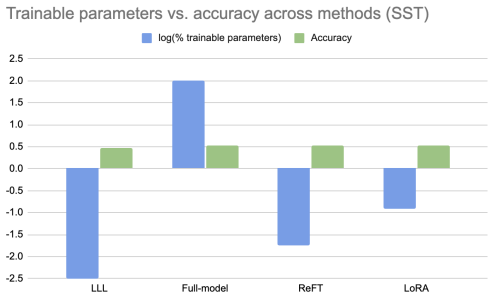


Figure 2: Visualizing efficiency (log(% trainable parameters) and accuracy tradeoff on SST)

Interestingly, we see a 2,467 MB reduction in GPU use as well as a dramatic decrease in trainable parameters, from 100% to 0.0178% (Figure 1) between Full-model and ReFT on SST, while ReFT gained accuracy ($0.520 \rightarrow 0.526$), suggesting that ReFT is more efficient on SST while being more suitable for the SST data, avoiding overfitting as full fine-tuning might. In general, both LoRA and ReFT achieved competitive performance as compared to full-fine tuning while using considerably fewer trainable parameters and compute.

Although full-model outperforms the other methods in accuracy on the CFIMDB dataset, we see a dramatic increase in memory for peak GPU memory for the full-model, similarly seen on SST (shown in Figure 2), showing a clear tradeoff between downstream accuracy and computational efficiency.

5.1.1 Paraphrase Detection:

For paraphrase detection, we first trained a full-model fine-tuning baseline on the complete Quora paraphrase training set to establish a strong performance benchmark. We then repeated experiments in a reduced data setting using a 25% subset of the training data to compare parameter-efficient adaptation methods under limited supervision. Using this subset, we evaluated ReFT, LoRA, and soft prompt tuning using the same development split for fair comparison.

Table 7: Paraphrase detection results on the Quora development and test sets.

Method	**Key Hyperparameters**	**Dev Accuracy**	**Test Accuracy**
Full-model fine-tuning	epochs=10, batch=8, lr=1×10^{-5}	**0.898**	**0.859**
ReFT	rank=4, layers=5, epochs=10, lr=1×10^{-4}	0.817	–
LoRA	rank=8, alpha=16, epochs=10, lr=1×10^{-4}	0.856	–
Soft Prompting	prompt length=20, epochs=10, lr=1×10^{-5}	0.743	–

Table 7 reports development accuracy across methods. Full-model fine-tuning achieved the strongest performance, while the parameter-efficient adaptations showed varying degrees of effectiveness on the reduced subset. Although these methods achieved development accuracies in the range of 0.743 to 0.817, all performed noticeably below the full-model baseline. This suggests limits to how much performance can be recovered through limited-parameter adaptation alone in this setting.

5.1.2 Sonnet Generation

We first tuned the full-model baseline and then explored whether decoding and reranking could improve performance on the held-out development prompts. The best-performing baseline used a learning rate of 2×10^{-5}, which achieved the strongest development chrF among the single-sample runs. We then found that decoding choices had a larger impact than additional parameter-efficient adaptation: best-of-N reranking with lightweight repetition and line-count penalties improved the score further, yielding our best overall development chrF of approximately 43.11. Our resulting test accuracy was 42.220.

To understand the contribution of reranking, we additionally compared several decoding variants on top of the best full-model setting. Best-of-10 reranking gave the strongest performance, while both

best-of-5 and best-of-15 performed worse. This suggests that a moderate candidate pool provides the best balance between diversity and selection quality: too few samples limit the chance of finding a strong candidate, while too many can introduce lower-quality outputs that make reranking less effective (see appendix figure 3).

Table 8: DPO sonnet generation results on held-out development prompts.

Corruption Strategy	**Key Hyperparameters**	**Dev chrF**
Line shuffling after truncation	$\beta = 0.1$, lr=1×10^{-5}, epochs=10	41.30
Partial continuation replacement	$\beta = 0.1$, lr=1×10^{-5}, epochs=10	41.04
Full continuation replacement (prompt fixed)	$\beta = 0.1$, lr=1×10^{-5}, epochs=10	41.27

Overall, DPO-based adaptation did not improve development chrF relative to either the strongest full-model baseline or the decoding-time reranking strategies. Across corruption variants, scores remained in a narrow range of 41.04 to 41.30 and consistently fell below the best reranked configuration. Line shuffling after truncation produced the strongest chrF among the DPO settings, while partial and full continuation replacement performed slightly worse. These results suggest that, in our setup, preference optimization with the tested synthetic corruptions was less effective than decoding-level structural guidance for improving sonnet generation quality.

6 Analysis

6.1 Analysis of ReFT, LoRA in comparison to baseline methods

For the comparison between ReFT, LoRA, full fine-tuning, and last linear layer, to account for randomness and address significance, we computed the mean and sample standard deviation (ddof=1) of the accuracies to summarize the variability of each method's performance across five random seeds. To assess whether observed differences were statistically significant, we performed paired t-tests comparing each method against full fine-tuning, using a threshold of $p < 0.05$ to determine significance.

Table 9: Significance for methods on sentiment analysis

Method	**SST (Mean $\pm$ Std)**	**CFIMDB (Mean $\pm$ Std)**
ReFT (r=4, layer=2)	0.5124 ± 0.0046	0.9696 ± 0.0040
ReFT (r=4, layer=1,6,11)	0.5148 ± 0.0078	0.9670 ± 0.0057
LoRA	$\mathbf{0.5216} \pm 0.0053$	0.9666 ± 0.0135
Full fine-tuning	0.5080 ± 0.0105	$\mathbf{0.9808} \pm 0.0018$
Last linear layer	0.4630 ± 0.0060	0.8586 ± 0.0108

P-values for methods on SST:

- **LoRA vs Full:** p-value = 0.0411 $\rightarrow$ Significant
- **ReFT-L2 vs Full:** p-value = 0.4910 $\rightarrow$ Not significant
- **ReFT-3Layers vs Full:** p-value = 0.2353 $\rightarrow$ Not significant
- **LLL vs Full:** p-value = 0.0001 $\rightarrow$ Significant

P-values for methods on CFIMDB:

- **LoRA vs Full:** p-value = 0.0653 $\rightarrow$ Not significant
- **ReFT-L2 vs Full:** p-value = 0.0070 $\rightarrow$ Significant
- **ReFT-3Layers vs Full:** p-value = 0.0041 $\rightarrow$ Significant
- **LLL vs Full:** p-value < 0.0001 $\rightarrow$ Significant

(ReFT-R2 refers to ReFT on rank = 4, layer = 2, and ReFT-3Layers refers to ReFT on rank 4, layers 1, 6, 11.)

Through this significance test, we confirm that on the SST dataset, PEFT methods (LoRA and ReFT) achieve comparable or slightly better performance than full fine-tuning while using significantly fewer trainable parameters. LoRA achieves a significantly higher performance on SST than the full fine-tuning model (0.5216 accuracy vs. 0.5080, with a p-value of 0.0411), while using significantly fewer parameters (only 0.1209%) as well as GPU (2861.55 MB compared to 4865.96). On the other hand, while the accuracy advantage ReFT had over full fine-tuning was not found to be significant, it does reveal that while training only 0.0178% of parameters and using 2398.49 MB of GPU memory, ReFT nevertheless achieved results that are statistically indistinguishable from training the entire 125M parameter model. Interestingly, on top of higher performance on SST, the standard deviations for PEFT (ReFT/LoRA) are generally lower than for the Full Model (especially on SST, 0.0046 vs 0.0105), suggesting that full-model updates can be seed-dependent and less stable.

In contrast, on the CFIMDB dataset, full fine-tuning achieves the significantly highest accuracy (0.9808), suggesting that larger or more complex inputs may benefit from adapting the full parameter space. However, LoRA did manage to statistically "keep up" with the Full model on the CFIMDB task, even if its mean was slightly lower. Nevertheless, LoRA's standard deviation is the highest on the CFIMBD task whereas full fine-tuning's is the lowest, an almost complete reversal from variance on SST, suggesting that LoRA is more sensitive to random initialization and changes in data. Since LoRA modifies the attention weights, an unideal initialization can disrupt the model's ability to attend to long-range dependencies in those long reviews. ReFT, by intervening on the representations, seems to be more stable, and doesn't fluctuate as much between runs.

Finally, although last linear layer uses the least amount of resources (0.0031% of parameters for both SST and CFIMDB, as well as only 681.36 MB and 591.18 MB for SST and CFIMDB, respectively), the accuracy tradeoff is significant across both datasets, with an accuracy of 0.4630 compared to 0.5080 for SST, and a dramatic drop in accuracy, with an accuracy of 0.8586 compared to 0.9808 for CFIMDB, highlighting the importance of adapting internal representations of the pretrained model.

Overall, however, due to limited compute, we acknowledge the limitations of significance analysis on five seeds; given more compute and time, we would ideally test across more seeds.

6.2 Qualitative Analysis of Sonnet Generation

To get a better understanding of sonnet generation beyond corpus-level chrF, we inspected representative continuations from the baseline and DPO-trained systems on held-out development prompts. In general, the strongest non-DPO generations preserved the Shakespearean register for several lines, even when they later became semantically loose or repetitive. In contrast, DPO-trained models more often drifted away from poetic continuation, introducing prose-like narration, repeated fragments, and adding unrelated content.

A common DPO failure mode was genre drift. For instance, the prompt beginning "Thine eyes I love, and they, as pitying me," one DPO continuation shifted into historical narration involving "Claudius" and "the Tiber," while another introduced a chapter-style heading ("CHAPTER XXI"). These outputs have several coherent sections, but they no longer resemble sonnet continuations in voice, structure, or topic. On the other hand, the baseline continuation for similar prompts more often stayed within the semantic field of love, sorrow, and beauty.

DPO outputs also showed more text corruption artifacts, including repeated tokens, symbol sequences, numbered fragments, and even a generated URL in one full-continuation replacement run. This suggests that the corruption-based preference pairs did not align well with the qualities most important for sonnet generation, such as poetic coherence, line structure, and consistency in style.

Across DPO variants, full continuation replacement produced the strongest degradation, while line-shuffling sometimes preserved short stretches of local fluency before drifting. Overall, these examples support our quantitative finding that decoding time reranking improved sonnet quality more reliably than DPO in this setup.

7 Conclusion

We evaluated several adaptation strategies for sentiment classification, paraphrase detection, and conditional sonnet generation, with a particular focus on lightweight extensions beyond standard

fine-tuning. Our results show that parameter-efficient methods can be competitive with full-model fine-tuning in classification settings, especially when the intervention type and placement are chosen carefully. For sonnet generation, the largest gains came from decoding and reranking rather than additional parameter-efficient training objectives, with best-of-N sampling and lightweight penalties producing the strongest chrF results. Overall, these findings suggest that effective downstream adaptation depends less on modifying the entire model and more on selecting targeted intervention strategies that match the task. Future work could explore stronger preference construction for DPO and broader comparisons of representation-level, weight-level, and decoding-time adaptation methods, and with greater compute, could utilize more robust significance testing.

8 Team contributions

Kristine: Worked on the core GPT model; implemented ReFT; ran experiments on and analyzed ReFT and LoRA; conducted seed-based and significance analysis for sentiment; contributed to the write-up.

Varsha: Worked on the core GPT model; implemented the sentiment classification code, LoRA, and DPO; analyzed DPO for sonnet generation; contributed to the write-up.

Isaias: Implemented paraphrase detection; implemented and improved sonnet generation; implemented soft prompting and hyperparameter tuning; contributed to the write-up.

A Appendix

Table 10: ReFT Performance across Parameters and Datasets

Parameters	SST Accuracy	CFIMDB Accuracy
rank 4, layer 5	0.504	0.967
rank 2, layer 5	0.511	0.959
rank 8, layer 5	0.509	0.967
rank 16, layer 5	0.517	0.955
rank 4, layers 2, 5, 8	0.525	0.967
rank 4, layer 2	0.507	**0.976**
rank 4, layer 10	0.473	0.959
rank 4, layer 0 (embedding)	0.509	0.967
rank 4, layer 11 (final)	0.48	0.873
rank 4, layers 3, 4, 5	0.51	0.955
rank 4, layers 0, 1, 2	0.507	0.963
rank 4, layers 1, 6, 11	**0.526**	0.971
rank 8, layers 1, 6, 11	0.525	0.951
rank 4, layers 8, 9, 10	0.507	**0.976**

Figure 3: Sonnet Generation: Development chrF by Experiment Category

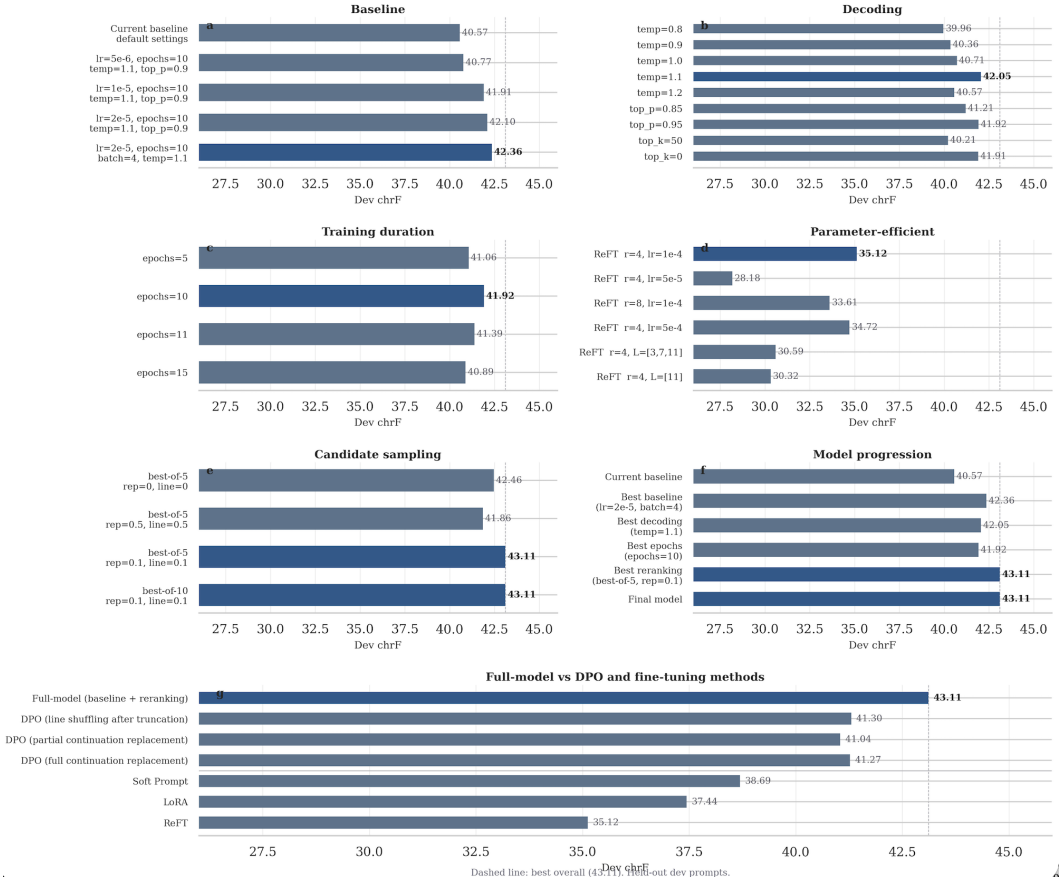


Table 11: Model Parameters and Efficiency for LoRA on Sentiment

Model	**rank 8, alpha 16**	**rank 4, alpha 8**	**rank 8, alpha 32**	**rank 16, alpha 32**
SST				
trainable parameters	298,757	151,301	298,757	593,669
total parameters	125,329,157	125,181,701	125,329,157	125,624,069
efficiency	0.2384%	0.1209%	0.2384%	0.4726%
peak GPU usage (MB)	2868.54	2861.55	2868.54	2873.15
CFIDMB				
trainable parameters	296,450	148,994	296,450	591,362
total parameters	125,326,850	125,179,394	125,326,850	125,621,762
efficiency	0.2365%	0.1190%	0.2365%	0.4707%
peak GPU usage (MB)	1565.04	1544.81	1565.04	1550.22

Table 12: Model Parameters and Efficiency for ReFT on Sentiment

Dataset	Method	Trainable Params	Total Params	Percent (%)	Peak GPU (MB)
SST	Full Model	125,031,938	125,031,938	100.00	4865.96
	Last Linear Layer	3,845	125,034,245	0.0031	681.36
	Rank 4	9,989	125,040,389	0.0080	1684.39
	Rank 4 (layers 2,5,8)	22,277	125,052,677	0.0178	2237.09
	Rank 4 (layer 2)	9,989	125,040,389	0.0080	2209.28
	Rank 4 (layer 10)	9,989	125,040,389	0.0080	810.30
	Rank 4 (layer 0)	9,989	125,040,389	0.0080	2559.28
	Rank 4 (layers 1,6,11)	22,277	125,052,677	0.0178	2398.49
CFIMDB	Full Model	125,034,245	125,034,245	100.00	2710.30
	Last Linear Layer	1,538	125,031,938	0.0012	591.18
	Rank 4	7,682	125,038,082	0.0061	1032.63
	Rank 4 (layers 2,5,8)	19,970	125,050,370	0.0160	1275.26
	Rank 4 (layer 2)	7,682	125,038,082	0.0061	1285.82
	Rank 4 (layer 10)	7,682	125,038,082	0.0061	614.02
	Rank 4 (layer 0)	7,682	125,038,082	0.0061	1452.43
	Rank 4 (layers 1,6,11)	19,970	125,050,370	0.0160	1354.35

References

[1] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. *OpenAI Technical Report*, 2018.

[2] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 1(8):9, 2019.

[3] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021.

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[5] Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. Reft: Representation finetuning for language models. *arXiv arXiv:2404.03592*, 2024.

[6] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2023.

[7] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[9] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, 2013.